

SUZUMA ACADEMY

MANUAL DOCENTE

Creación de Agentes de IA — De lo básico a lo intermedio

Guía de apoyo para impartición, prácticas, evaluación y proyecto integrador

Cómo utilizar este manual

Este manual reúne y organiza el material desarrollado para el curso de creación de agentes de IA de SuZuMa Academy. Está pensado como apoyo del docente durante la explicación, práctica, seguimiento y evaluación.

• Usar las diapositivas como apoyo visual y este manual como guía de conducción.

• Priorizar la práctica: cada concepto debe terminar en una pequeña prueba, discusión o implementación.

• Conservar la progresión del curso: conversación → memoria → herramientas → RAG → planificación → multiagente → evaluación y despliegue.

• No exigir arquitectura compleja cuando una solución más sencilla resuelva correctamente el problema.

• Evaluar tanto el resultado como el proceso: decisiones, pruebas, límites, seguridad y capacidad de explicar el sistema.

Mapa general del curso

Etapa	Módulo	Competencia central
1	Fundamentos	Comprender qué es un agente y cómo se diferencia de un chatbot.
2	Prompt Engineering	Diseñar instrucciones claras, estructuradas y verificables.
3	Primer agente con Python	Construir un agente inicial programable.
4	Memoria y contexto	Gestionar historial, estado y memoria.
5	Herramientas / Tool Calling	Conectar el agente con funciones y servicios.
6	RAG	Consultar información propia mediante recuperación documental.
7	Planificación y orquestación	Descomponer objetivos y coordinar pasos.
8	Multiagente	Coordinar especialistas con responsabilidades diferenciadas.
9	Evaluación y despliegue	Medir, observar, asegurar y operar el sistema.
Final	Proyecto integrador	Construir y demostrar un agente completo.

Módulo 1 — Fundamentos de IA y agentes

Introducción conceptual para establecer el lenguaje común del curso y distinguir conversación, contexto, herramientas y agentes.

Objetivos docentes

Que el estudiante pueda explicar qué problema resuelve un agente, qué componentes puede incorporar y por qué la arquitectura debe partir de una necesidad.

Actividad sugerida

Plantear una tarea cotidiana o académica y pedir dos instrucciones: una libre y otra estructurada. Comparar resultados y discutir qué cambió y por qué.

Módulo 2 — Prompt Engineering

Diseño de instrucciones para conseguir respuestas consistentes, útiles y verificables.

Objetivos docentes

Que el estudiante pueda estructurar instrucciones con objetivo, contexto, restricciones, formato de salida y criterios de calidad.

Actividad sugerida

Plantear una tarea cotidiana o académica y pedir dos instrucciones: una libre y otra estructurada. Comparar resultados y discutir qué cambió y por qué.

Módulo 3 — Primer Agente con Python

1. MÓDULO 3
PRIMER AGENTE
CON PYTHON
Del prompt al código

PYTHON

API

LLM

AGENTE

> *Nota para el docente: Presenta el salto del diseño al desarrollo. El estudiante ya tiene un agente diseñado y un prompt. Ahora aprenderá a conectarlo con Python y un modelo mediante una API.*

2. ¿QUÉ VAMOS A CONSTRUIR?

Un agente mínimo, funcional y comprobable.

02

USUARIO

PYTHON

API

LLM

RESPUESTA

Primera meta: enviar una instrucción y recibir una respuesta desde Python.

> *Nota para el docente: Explica que esta primera versión será deliberadamente pequeña. Primero hacemos funcionar la comunicación básica; después añadiremos memoria, herramientas y RAG.*

3. OBJETIVOS DEL MÓDULO

Al terminar, el estudiante podrá...

03

Preparar un entorno Python para IA.

Gestionar una API Key de forma segura.

Enviar instrucciones a un modelo mediante una API.

Interpretar una respuesta JSON.

Construir una conversación básica.

Manejar errores comunes.

> *Nota para el docente: Presenta los resultados. Señala que seguridad de credenciales y manejo de errores forman parte del desarrollo profesional, no son detalles opcionales.*

4. REQUISITOS

Antes de programar

04

Python

VS Code

Terminal

Git

Cuenta/API

Necesitamos una credencial para comunicarnos con el servicio de IA.

Nunca la escribas directamente en el código.

> *Nota para el docente: Ajusta los proveedores de API a la plataforma elegida por el curso. El principio es general: necesitamos una credencial y debemos almacenarla fuera del código fuente.*

5. ESTRUCTURA DEL PROYECTO

Empecemos simple

05

agente_ia/

■■■ .env

■■■ .gitignore

■■■ requirements.txt

■■■ agente.py

.env → credenciales

requirements → dependencias

agente.py → lógica

gitignore → secretos

> *Nota para el docente: Explica cada archivo. El estudiante debe entender que separar configuración, dependencias y código facilita mantenimiento y evita exponer secretos.*

6. ENTORNO VIRTUAL

Aislar las dependencias del proyecto

06

python -m venv .venv

activar en macOS/Linux

source .venv/bin/activate

activar en Windows

.venv\Scripts\activate

¿Por qué?

Evita mezclar dependencias entre proyectos.

> *Nota para el docente: Demuestra el comando correspondiente al sistema operativo del laboratorio. Verifica que el entorno esté activo antes de instalar dependencias.*

7. DEPENDENCIAS

Instalar solo lo que necesitamos

07

pip install python-dotenv

pip freeze > requirements.txt

Objetivo

Reproducir el entorno del proyecto.

El SDK concreto del proveedor de LLM se incorporará aquí según la API elegida.

> *Nota para el docente: No fijes una biblioteca concreta si el curso pretende ser independiente del proveedor. Presenta dotenv como dependencia común y después instala el SDK que corresponda al servicio seleccionado.*

8. API KEY

Una credencial no pertenece al código

08

```
# .env

AI_API_KEY=tu_clave_aqui

# agente.py

from dotenv import load_dotenv

import os

load_dotenv()

api_key = os.getenv("AI_API_KEY")
```

NO

No publiques la clave en GitHub.

> *Nota para el docente: Explica el riesgo de publicar una API Key. Muestra .gitignore y comprueba que .env no quede incluido en el repositorio.*

9. .GITIGNORE

La primera barrera contra un secreto publicado

09

```
.venv/

.env

__pycache__/

*.pyc
```

Regla

Los secretos se gestionan fuera del código fuente.

Comprueba el repositorio antes de hacer push.

> *Nota para el docente: Haz que el estudiante compruebe git status y confirme que .env no aparece entre los archivos que serán enviados.*

10. ¿QUÉ ES UNA API?

La puerta de comunicación entre nuestro programa y un servicio

10

TU PROGRAMA

API

SERVICIO IA

Solicitud → procesamiento → respuesta

Nota para el docente: Explica API como contrato de comunicación. El estudiante no necesita dominar redes para esta sesión, pero sí entender que envía una solicitud y recibe una respuesta estructurada.

11. JSON

El formato que encontraremos en la comunicación

11

{

"role": "assistant",

"content": "Hola, ¿en qué puedo ayudarte?"

}

JSON

Datos estructurados que nuestro programa puede leer y procesar.

No necesitamos memorizar cada campo: aprenderemos a inspeccionar la respuesta.

Nota para el docente: Muestra una respuesta JSON real del proveedor elegido cuando hagas la práctica. Explica que la estructura exacta puede variar entre APIs; el concepto de datos estructurados permanece.

12. NUESTRA PRIMERA LLAMADA

El objetivo: obtener una respuesta desde Python

12

mensaje = "Explica qué es una API a un estudiante de TICs."

respuesta = cliente.modelo(...)

print(respuesta)

PROMPT

API

LLM

RESPUESTA

Nota para el docente: El código es deliberadamente pseudocódigo en esta diapositiva porque el cliente exacto depende del proveedor. En el laboratorio se sustituye por el SDK oficial elegido para el curso.

13. SEPARAMOS RESPONSABILIDADES

Configuración ≠ lógica ≠ conversación

13

CONFIGURACIÓN

API Key y modelo

LÓGICA

Funciones Python

CONVERSACIÓN

Mensajes y contexto

Esta separación facilita probar, modificar y mantener el agente.

> *Nota para el docente: Introduce una pequeña disciplina de arquitectura. El estudiante debe evitar meter API Keys, prompts, lógica y salida todo mezclado en una sola línea.*

14. CONSTRUIMOS SUZUMA TUTOR V1

Primera versión funcional

14

SYSTEM = """Eres SuZuMa Tutor,

un tutor universitario de Python.

Explica con claridad y ejemplos."""

USER = input("Tú: ")

enviar SYSTEM + USER al modelo

imprimir respuesta

V1

Una versión pequeña que ya conversa.

> *Nota para el docente: Aquí el estudiante toma su prompt del módulo anterior y lo convierte en una constante o configuración del programa. El código real de llamada al SDK se implementará según el proveedor seleccionado.*

15. DE UNA PREGUNTA A UNA RESPUESTA

El flujo completo

15

input()

mensaje

API

LLM

respuesta

print()

USUARIO → PROGRAMA → MODELO → PROGRAMA → USUARIO

> *Nota para el docente: Haz que los estudiantes tracen el recorrido de una pregunta. Esto les ayudará a depurar posteriormente: si algo falla, deben saber en qué tramo ocurrió.*

16. CONVERSACIÓN

Una respuesta aislada todavía no es una conversación

16

messages = [

{"role": "system", "content": SYSTEM},

{"role": "user", "content": "¿Qué es una variable?"}

]

SYSTEM

USER

ASSISTANT

Historial = contexto

Nota para el docente: Explica los roles system, user y assistant. Aclara que mantener mensajes previos permite continuidad conversacional. La memoria persistente se estudiará después.

17. BUCLE DE CONVERSACIÓN

Ahora el programa puede seguir preguntando

17

while True:

user = input("Tú: ")

if user.lower() == "salir":

break

enviar historial al modelo

mostrar respuesta

INPUT

LLM

OUTPUT

Historial acumulado

Nota para el docente: Explica el bucle. Señala que un agente conversacional necesita conservar el historial que se envía al modelo. En esta etapa la memoria vive en el proceso; todavía no es una base persistente.

18. MANEJO DE ERRORES

Un agente real debe saber fallar

18

try:

respuesta = llamar_modelo(mensajes)

except Exception as e:

print("Error al consultar el servicio")

print(e)

Clave inválida

Sin internet

Límite de uso

Respuesta inesperada

Nota para el docente: Explica que los errores son parte normal del desarrollo. No ocultes todos los detalles al desarrollador, pero sí proporciona un mensaje comprensible al usuario. Introduce try/except como mecanismo básico.

19. LABORATORIO 03

Construye tu primer agente con Python · 30 minutos

19

Crear carpeta y entorno virtual.

Configurar .env y .gitignore.

Instalar dependencias.

Conectar el SDK/API elegido.

Enviar el System Prompt del Módulo 2.

Recibir e imprimir la respuesta.

Agregar conversación básica.

30

MIN

> *Nota para el docente: Este es el laboratorio central. El instructor debe trabajar en paralelo en pantalla y verificar que cada estudiante pueda ejecutar una primera consulta antes de avanzar.*

20. RETO · MODIFICA EL AGENTE

No copies: cambia su comportamiento

20

Convierte SuZuMa Tutor en uno de estos agentes:

Tutor de bases de datos

Asistente de redes

Revisor de código Python

Soporte técnico universitario

Cambia rol + objetivo + ejemplo + tono.

> *Nota para el docente: El objetivo del reto es demostrar que comprendieron la separación entre instrucciones y código. Deben poder reutilizar la misma estructura de programa cambiando el comportamiento del agente.*

21. PRUEBA DE CALIDAD

¿Funciona realmente?

21

✓

¿La API responde?

✓

¿La API Key está fuera del código?

✓

¿El System Prompt se aplica?

✓

¿El historial funciona?

✓

¿Los errores se controlan?

✓

¿El programa termina correctamente?

Nota para el docente: Utiliza esta lista antes de considerar terminado el laboratorio. El estudiante debe aprender a probar sistemáticamente y no solo celebrar que una respuesta apareció.

22. MINI EVALUACIÓN

Comprueba que entiendes el flujo

22

1

¿Qué función cumple una API?

2

¿Por qué usamos .env?

3

¿Qué representa el System Prompt?

4

¿Por qué necesitamos historial?

5

¿Qué ocurre si la API devuelve un error?

Nota para el docente: Evalúa comprensión conceptual y no memorización de comandos. El estudiante debe poder explicar el flujo aunque cambie el SDK.

23. ENTREGABLE

Tu primer agente funcional

23

Código Python

.env seguro

Conversación

Entrega mínima

Repositorio + README + captura de una conversación funcionando

Nota para el docente: Define el entregable. El README debe indicar cómo instalar dependencias, configurar la API Key y ejecutar el agente, sin incluir secretos.

24. TAREA

Evoluciona tu agente

24

Añade una instrucción nueva.

Agrega una restricción.

Cambia el formato de respuesta.

Prueba tres preguntas diferentes.

Registra un error y cómo lo resolviste.

Actualiza el README.

V1

V2

Nota para el docente: La tarea prepara el siguiente paso: mejorar el agente a partir de pruebas. Pide que documenten qué cambiaron y qué efecto observaron.

25. YA TIENES TU PRIMER AGENTE

El siguiente paso es darle memoria y contexto persistente.

25

PYTHON

API

LLM

AGENTE

MEMORIA

MÓDULO 4 · MEMORIA Y CONTEXTO

Hoy: conectamos Python con un modelo. Después: haremos que el agente recuerde.

Nota para el docente: Cierra mostrando la evolución. El estudiante ya tiene un agente básico. El siguiente módulo introducirá memoria y contexto para convertir la conversación en un sistema más útil.

Cierre y evidencia sugerida

Pedir al estudiante una explicación breve del concepto central, una demostración práctica y una evidencia verificable (código, prueba, diagrama, resultado o registro).

Módulo 4 — Memoria y Contexto

1. MÓDULO 4
MEMORIA
Y CONTEXTO
Cómo hacer que un agente mantenga información útil

MENSAJE

CONTEXTO

MEMORIA

RESPUESTA

> *Nota para el docente: Presenta el problema: en el módulo anterior el agente podía conversar, pero cada ejecución tenía límites de contexto. Hoy aprenderemos a manejar historial, contexto de usuario y memoria persistente.*

2. EL PROBLEMA

Una conversación sin contexto pierde continuidad.

02

Usuario

Agente

“Mi nombre es Ana y estudio TICs.”

“¿Cómo puedo ayudar?”

Pregunta clave: ¿cómo hacemos que el sistema conserve información útil?

> *Nota para el docente: Usa un ejemplo sencillo de pérdida de continuidad. Aclara que el problema puede deberse a que el historial no se envió, a límites de contexto o a que nunca se diseñó una memoria persistente.*

3. OBJETIVOS DEL MÓDULO

Al terminar podrás...

03

Distinguir contexto, historial, estado y memoria.

Mantener una conversación mediante mensajes acumulados.

Definir qué información merece conservarse.

Guardar información estructurada.

Recuperar memoria relevante para responder.

Diseñar una memoria con límites y reglas claras.

> *Nota para el docente: Aclara que 'memoria' no significa guardar todo. El objetivo es diseñar qué información se conserva, durante cuánto tiempo y para qué se recupera.*

4. CUATRO CONCEPTOS

No son exactamente lo mismo

04

HISTORIAL

Mensajes de la conversación

CONTEXTO

Información disponible para una respuesta

ESTADO

Datos que describen la situación actual

MEMORIA

Información conservada para uso posterior

Nota para el docente: Diferencia los cuatro conceptos. Puedes usar una libreta como analogía: historial es lo que ocurrió en la conversación; estado es cómo está ahora el proceso; memoria es lo que decidimos conservar para futuras interacciones.

5. HISTORIAL DE CONVERSACIÓN

La forma más sencilla de mantener continuidad

05

```
messages = [
{"role":"system", "content": SYSTEM},
{"role":"user", "content":"Mi nombre es Ana."},
{"role":"assistant", "content":"Mucho gusto, Ana."},
{"role":"user", "content":"¿Qué recuerdas de mí?"}
]
```

El historial viaja nuevamente al modelo cuando necesitamos contexto.

Nota para el docente: Explica que el modelo normalmente no 'recuerda' por sí mismo entre llamadas. Nuestra aplicación conserva y reenvía mensajes relevantes. La forma exacta de manejar el historial depende del SDK y del proveedor.

6. EL PROBLEMA DEL HISTORIAL

Más contexto no siempre significa mejor contexto

06

10 mensajes

100 mensajes

¿TODO?

Costo · latencia · límites de contexto · ruido

Necesitamos seleccionar lo relevante.

Nota para el docente: Introduce la idea de que enviar todo el historial puede ser costoso o poco útil. Esto prepara el concepto de resumen, selección de contexto y memoria estructurada.

7. CONTEXTO DE USUARIO

Información que ayuda a personalizar

07

Nombre

Ana

Carrera

Ingeniería de Software

Nivel

Intermedio

Tema

Python

Preferencia

Ejemplos prácticos

Contexto útil → respuestas más adaptadas

> *Nota para el docente: Haz énfasis en relevancia. No toda la información del usuario merece conservarse. El diseño debe comenzar por qué datos son necesarios para la tarea.*

8. ESTADO

Información que describe la situación actual

08

```
state = {
"sesion_activa": True,
"tema": "funciones",
"ejercicio": 3,
"intentos": 2,
"nivel": "básico"
}
```

ESTADO

¿Qué está ocurriendo ahora?

Puede cambiar durante la sesión.

> *Nota para el docente: Compara estado con memoria. El ejercicio actual, el paso en el que está el usuario o una sesión activa son datos de estado. Pueden desaparecer cuando termina la tarea.*

9. MEMORIA

Lo que decidimos conservar

09

INTERACCIÓN

EXTRACCIÓN

MEMORIA

FUTURO

Memoria = información seleccionada y reutilizable

> *Nota para el docente: Introduce memoria como una decisión de diseño. El sistema debe identificar información que podría ser útil más adelante y guardarla en una forma que pueda recuperarse.*

10. ¿QUÉ DEBERÍA GUARDAR?

Diseñemos una memoria útil

10

Preferencia estable

Prefiere ejemplos prácticos

Perfil académico

Estudia Ingeniería de Software

Objetivo

Está aprendiendo Python

Dato irrelevante

Comentarios sin utilidad

Secreto

API Key

Todo

Cada mensaje completo

Pregunta guía: ¿será útil en una conversación futura?

> *Nota para el docente: Discute criterios de utilidad. No conviertas la memoria en un almacén indiscriminado. Los secretos nunca deben tratarse como memoria de usuario.*

11. MEMORIA ESTRUCTURADA

Guardar datos con una forma que podamos consultar

11

memory = {

"profile": {

"career": "Ingeniería de Software",

"level": "intermedio"

},

"preferences": {

"style": "ejemplos prácticos"

}

}

Ventajas

Fácil de leer

Fácil de modificar

Fácil de consultar

Puede persistirse

> *Nota para el docente: Presenta JSON o diccionarios como una estructura didáctica. No es todavía una solución de producción; es una forma de enseñar cómo representar memoria.*

12. MEMORIA EN ARCHIVO

Primera forma de persistencia

12

import json

with open("memory.json", "w") as f:

json.dump(memory, f, indent=2)

with open("memory.json") as f:

memory = json.load(f)

PROCESO

Programa↓Archivo↓Programa

Nota para el docente: Explica persistencia con un archivo JSON. Es suficiente para aprender el concepto. En aplicaciones reales pueden utilizarse bases de datos o almacenes especializados.

13. RECUPERAR MEMORIA

No basta con guardar: hay que usarla

13

MENSAJE

BUSCAR MEMORIA

CONTEXTO

LLM

Guardar → recuperar → incorporar al contexto → responder

Nota para el docente: Esta es la arquitectura básica de recuperación. Explica que una memoria no tiene valor si nunca se incorpora de manera relevante al contexto de la respuesta.

14. EJEMPLO

SuZuMa Tutor recuerda una preferencia

14

ANTES

“Explícame listas en Python.”

Respuesta genérica

DESPUÉS

Memoria: prefiere ejemplos prácticos.

Respuesta adaptada

Nota para el docente: Usa un ejemplo concreto para mostrar el valor de memoria. La memoria no debe cambiar arbitrariamente el contenido técnico; debe ayudar a adaptar la forma de la respuesta.

15. MEMORIA NO ES LO MISMO QUE HISTORIAL

Dos capas, dos propósitos

15

HISTORIAL

“¿Qué es una lista?”“Es una colección...”“Dame un ejemplo...”

Contexto inmediato

MEMORIA

“Prefiere ejemplos prácticos.”

Utilidad futura

Nota para el docente: Esta distinción debe quedar clara. El historial representa conversación; la memoria representa información seleccionada que puede ser útil más adelante.

16. DISEÑO DE MEMORIA

Reglas antes de implementar

16

¿Qué información se guarda?

¿Quién puede verla?

¿Cuánto tiempo se conserva?

¿Cuándo se actualiza?

¿Cuándo deja de ser relevante?

¿Cómo se elimina o corrige?

La memoria es una decisión de producto y de arquitectura.

Nota para el docente: Introduce criterios de diseño y privacidad de forma general. La memoria debe ser limitada, explicable y controlable. No guardes secretos ni información innecesaria.

17. LABORATORIO 04

Añade memoria a SuZuMa Tutor · 30 minutos

17

Crear memory.json.

Definir perfil y preferencias.

Guardar información estructurada.

Cargar memoria al iniciar.

Seleccionar memoria relevante.

Incorporarla al contexto del modelo.

Probar dos conversaciones diferentes.

30

MIN

Nota para el docente: Durante el laboratorio, comienza con JSON local. El objetivo es comprender el ciclo completo. No intentes introducir una base de datos todavía.

18. RETO

¿Qué memoria guardarías para estos usuarios?

18

A

Estudiante que prefiere ejemplos visuales

B

Usuario que compartió su API Key

C

Tema actual: estructuras de datos

D

Comentario irrelevante de una conversación

Clasifica: GUARDAR · NO GUARDAR · SOLO CONTEXTO

> *Nota para el docente: Discute cada caso. La API Key no debe guardarse como memoria de usuario. El tema actual puede ser estado o contexto. Una preferencia estable puede convertirse en memoria.*

19. PRUEBA DE CALIDAD

La memoria debe mejorar la experiencia, no estorbar

19

✓

¿La información guardada es relevante?

✓

¿Se recupera cuando hace falta?

✓

¿Se evita enviar información innecesaria?

✓

¿Puede actualizarse?

✓

¿Se excluyen secretos?

✓

¿El agente sigue respondiendo correctamente sin esa memoria?

> *Nota para el docente: Introduce una prueba importante: el agente no debería quedar inútil si una memoria no está disponible. La memoria debe ser una mejora, no un punto único de fallo.*

20. DE ARCHIVO A BASE DE DATOS

La misma idea puede escalar

20

memory.json

SQLite

PostgreSQL

Vector DB*

*Más adelante veremos recuperación semántica con RAG.

El concepto permanece: guardar → recuperar → usar.

> *Nota para el docente: Muestra la evolución tecnológica sin implementarla todavía. El objetivo es que el estudiante entienda que el patrón de memoria puede utilizar distintos almacenamientos.*

21. MINI EVALUACIÓN

Comprueba lo aprendido

21

1

¿Qué diferencia hay entre historial y memoria?

2

¿Qué es el estado?

3

¿Por qué no debemos guardar todo?

4

¿Cómo recuperamos memoria útil?

5

¿Qué información NO debe tratarse como memoria?

Nota para el docente: Evalúa comprensión conceptual. Pide ejemplos, no definiciones memorizadas.

22. ENTREGABLE

SuZuMa Tutor V2

22

V1

MEMORIA

CONTEXTO

V2

Debe conservar al menos:

perfil · preferencias · tema actual · memoria recuperable

Nota para el docente: Define el entregable mínimo. El estudiante debe demostrar que la memoria modifica de forma útil el contexto de una respuesta.

23. TAREA

Diseña las reglas de memoria de tu propio agente

23

Qué datos guardará.

Qué datos no guardará.

Cuándo se actualizarán.

Cómo se recuperarán.

Cuándo dejarán de ser relevantes.

Cómo se probará que funcionan.

DISEÑO

ANTESDECODIFICAR

Nota para el docente: Pide que documenten las reglas antes de programar. Esto refuerza el enfoque de arquitectura y evita guardar información indiscriminadamente.

24. LO QUE YA SABEMOS

El agente ahora tiene más contexto

24

PROMPT

PYTHON

LLM

HISTORIAL

MEMORIA

Próximo salto: darle herramientas para actuar.

MÓDULO 5 · HERRAMIENTAS Y TOOL CALLING

> *Nota para el docente: Conecta con el siguiente módulo. El agente ya puede trabajar con contexto y memoria. Ahora aprenderá a utilizar funciones externas.*

25. DE CONVERSAR A RECORDAR

El agente empieza a mantener continuidad

25

USUARIO

CONTEXTO

MEMORIA

LLM

RESPUESTA

Guardar → recuperar → contextualizar → responder

Siguiente: TOOLS · el agente podrá hacer cosas.

> *Nota para el docente: Cierre. Pide al estudiante que explique con sus palabras la diferencia entre historial, contexto, estado y memoria. La siguiente sesión introducirá herramientas y Tool Calling.*

Cierre y evidencia sugerida

Pedir al estudiante una explicación breve del concepto central, una demostración práctica y una evidencia verificable (código, prueba, diagrama, resultado o registro).

Módulo 5 — Herramientas y Tool Calling

1. MÓDULO 5
HERRAMIENTAS
Y TOOL CALLING
De responder a ejecutar acciones

LLM

DECISIÓN

TOOL

RESULTADO

Nota para el docente: Presenta el salto conceptual: el modelo puede identificar que necesita una herramienta y nuestra aplicación puede ejecutar una función.

2. ¿QUÉ ES UNA HERRAMIENTA?

Una capacidad externa que el agente puede invocar.

02

AGENTE

TOOL

FUNCIÓN

RESULTADO

Ejemplos: calcular · consultar · buscar · transformar · validar

Nota para el docente: Define herramienta como una función o capacidad que la aplicación puede ejecutar cuando el agente la necesita.

3. OBJETIVOS DEL MÓDULO

Al terminar podrás...

03

Diseñar funciones como herramientas de un agente.

Describir una herramienta con entradas y salida.

Entender el ciclo de Tool Calling.

Permitir que el modelo seleccione una herramienta.

Ejecutar la función desde Python.

Devolver el resultado al modelo.

Controlar errores y límites de ejecución.

Nota para el docente: Presenta el objetivo como una evolución del agente.

4. MODELO VS. HERRAMIENTA

No hacen exactamente lo mismo

04

MODELO

Interpreta lenguaje

Genera contenido

Decide qué necesita

Analiza el resultado

HERRAMIENTA

Ejecuta una acción

Consulta un sistema

Calcula

Devuelve datos

Nota para el docente: Una herramienta no sustituye al modelo; es una capacidad disponible para que el modelo la solicite.

5. PRIMERA TOOL

Una calculadora

05

```
def calcular(a: float, b: float, operacion: str):
if operacion == "sumar":
return a + b
if operacion == "restar":
return a - b
raise ValueError("Operación no permitida")
```

TOOL

Entradas↓Función↓Resultado

Nota para el docente: Explica que una herramienta comienza como una función normal de Python.

6. CONTRATO DE UNA HERRAMIENTA

Nombre + descripción + parámetros + salida

06

Nombre

Descripción

Parámetros

Resultado

El modelo necesita saber cuándo utilizarla y qué datos debe proporcionar.

Una herramienta mal descrita produce decisiones deficientes.

Nota para el docente: Introduce el concepto de esquema o contrato.

7. DESCRIPCIÓN DE TOOL

Debe ser clara y específica

07

Nombre: calcular

Descripción:

"Realiza operaciones aritméticas básicas.

Usarla cuando el usuario solicite un cálculo."

Parámetros:

- a: número

- b: número

- operacion: sumar | restar

MEJOR

Qué haceCuándo usarlaQué recibeQué devuelve

Nota para el docente: Compara una descripción vaga con una descripción útil.

8. TOOL CALLING

El modelo puede solicitar una función

08

USUARIO

LLM

TOOL CALL

PYTHON

RESULTADO

El modelo no ejecuta mágicamente la función: nuestra aplicación la ejecuta.

Nota para el docente: El modelo genera una solicitud estructurada; el programa recibe la solicitud, ejecuta la función y devuelve el resultado.

9. CICLO COMPLETO

Tool Calling paso a paso

09

1

Pregunta

2

Modelo decide

3

Tool Call

4

Python ejecuta

5

Resultado

6

Modelo responde

PREGUNTA → DECISIÓN → ACCIÓN → RESULTADO → RESPUESTA

Nota para el docente: Recorre los seis pasos con un ejemplo: calcular 125 por 48.

10. ¿CUÁNDO USAR UNA TOOL?

El agente debe decidir si realmente la necesita

10

SIN TOOL

“¿Qué es una API?”Puede responder directamente.

CON TOOL

“Calcula mi promedio.”Necesita datos o función.

Nota para el docente: No todo requiere una herramienta; una mala configuración puede provocar llamadas innecesarias.

11. SEGUNDA TOOL

Consultar una base de datos

11

def buscar_alumno(matricula: str):

consultar BD

return {

"nombre": "Ana López",

"carrera": "TICs",

"semestre": 4

}

TOOL

EntradamatrículaSalidadatos

Nota para el docente: Utiliza una función simulada si no existe una base de datos.

12. TERCERA TOOL

Consultar una API externa

12

AGENTE

API

SERVICIO

DATOS

El agente puede combinar LLM + herramientas + servicios externos.

Nota para el docente: Una herramienta puede ser un wrapper alrededor de una API.

13. SEGURIDAD DE LAS HERRAMIENTAS

No todo debe poder ejecutarse automáticamente

13

Permisos excesivos

Entradas no validadas

Acciones irreversibles

Datos sensibles

Principio: mínima capacidad necesaria + validación + límites.

Nota para el docente: Una herramienta convierte texto en capacidad de acción; requiere validación y permisos mínimos.

14. HERRAMIENTAS DE SUZUMA TUTOR

Diseñemos el conjunto inicial

14

calcular

buscar_alumno

consultar_materia

obtener_horario

OBJETIVO

Responder preguntas universitarias utilizando capacidades externas.

Nota para el docente: Presenta el conjunto de herramientas del proyecto.

15. DESCRIBIR LAS TOOLS

El modelo necesita conocer sus capacidades

15

tools = [

{

"name": "calcular",

"description": "Realiza cálculos básicos.",

"parameters": {

"a": "number",

"b": "number",

"operacion": "string"

}

}

]

ESQUEMA

NombreDescripciónParámetrosTipos

Nota para el docente: El formato exacto depende del proveedor; el principio es describir función y argumentos.

16. EL AGENTE DECIDE

No programamos cada conversación con if/else

16

ENFOQUE RÍGIDO

if "promedio" in texto: llamar_calculo()if "horario" in texto: consultar_horario()

ENFOQUE CON TOOLS

Describimos las herramientas y dejamos que el modelo seleccione la adecuada según la intención.

Nota para el docente: Explica la diferencia entre reglas rígidas y selección mediante herramientas.

17. LABORATORIO 05

Añade tres herramientas a tu agente · 35 minutos

17

Implementar calcular().

Implementar una consulta simulada.

Definir una tercera herramienta.

Describir cada tool para el modelo.

Conectar Tool Call → Python → resultado.

Probar al menos cinco preguntas.

Registrar cuándo el agente eligió mal.

35

MIN

Nota para el docente: Laboratorio central. Implementar tres funciones y conectar el ciclo Tool Call → Python → resultado.

18. RETO

Construye una herramienta nueva

18

Tu agente necesita una capacidad que no tenía.

Convertir unidades

Consultar materias

Validar un dato

Generar un reporte

Buscar en un documento

Consultar una API

Define: propósito · parámetros · resultado · límites

Nota para el docente: Cada estudiante crea una tool con propósito, parámetros, resultado y límites.

19. PRUEBAS

Una herramienta puede fallar

19

Entrada válida

Entrada faltante

Tipo incorrecto

Resultado vacío

Error de servicio

Permiso insuficiente

Nunca confíes ciegamente en los argumentos generados por el modelo.

Nota para el docente: Prueba entradas válidas, faltantes, tipos incorrectos y errores de servicio.

20. LÍMITES DEL AGENTE

Capacidad no significa libertad total

20

Tiempo máximo

Número de llamadas

Herramientas permitidas

Acciones sensibles

Diseñar límites es parte de construir un agente confiable.

Nota para el docente: Diseñar límites es parte de construir un agente confiable.

21. MINI EVALUACIÓN

Comprueba lo aprendido

21

1

¿Qué es una tool?

2

¿Qué información necesita el modelo para usarla?

3

¿Quién ejecuta realmente la función?

4

¿Por qué debemos validar parámetros?

5

¿Qué límites pondrías a una herramienta sensible?

Nota para el docente: Evalúa comprensión del flujo y seguridad.

22. ENTREGABLE

SuZuMa Tutor V3 · Agente con herramientas

22

V2

TOOLS

TOOL CALL

EJECUCIÓN

V3

Debe demostrar selección + ejecución + resultado.

Nota para el docente: Debe mostrar al menos dos herramientas funcionales y evidencia de selección y ejecución.

23. TAREA

Diseña una herramienta para tu proyecto final

23

Nombre de la herramienta.

Problema que resuelve.

Parámetros de entrada.

Tipo de cada parámetro.

Resultado esperado.

Errores posibles.

Permisos o límites.

Ejemplo de uso.

TOOL

DISEÑO→CÓDIGO

Nota para el docente: Documentar primero el contrato y después implementar la función.

24. LA ARQUITECTURA CRECE

Ya tenemos un agente que puede actuar

24

PROMPT

PYTHON

LLM

MEMORIA

TOOLS

RESULTADO

Próximo salto: permitir que el agente consulte documentos propios.

MÓDULO 6 · RAG Y AGENTES CON DOCUMENTOS

Nota para el docente: Conecta con RAG.

25. DE RESPONDER A ACTUAR

El agente ya puede utilizar capacidades externas

25

USUARIO

AGENTE

DECIDE

TOOL

RESULTADO

Interpretar → decidir → ejecutar → observar → responder

Siguiente: RAG · información propia para agentes.

Nota para el docente: Cierra reforzando que Tool Calling es el puente entre lenguaje y acción.

Cierre y evidencia sugerida

Pedir al estudiante una explicación breve del concepto central, una demostración práctica y una evidencia verificable (código, prueba, diagrama, resultado o registro).

Módulo 6 — RAG y agentes con documentos

1. MÓDULO 6

RAG y agentes con documentos

SuZuMaCyber Instructor

Nota para el docente: De responder con conocimiento general a responder con información propia y verificable.

Sugerencia didáctica: pedir al estudiante que explique el concepto con sus propias palabras y relacionarlo con SuZuMa Tutor.

2. Objetivos del módulo

Al terminar podrás:

• Explicar qué es RAG.

• Convertir documentos en conocimiento consultable.

• Dividir texto en fragmentos.

• Comprender embeddings y búsqueda semántica.

• Recuperar contexto relevante.

• Construir un agente que responda usando documentos.

SuZuMaCyber Instructor

Nota para el docente: Enfatiza que RAG no es una "memoria mágica": es un proceso de recuperación de información antes de generar la respuesta.

Sugerencia didáctica: pedir al estudiante que explique el concepto con sus propias palabras y relacionarlo con SuZuMa Tutor.

3. El problema

Un modelo puede saber mucho… pero no necesariamente conoce:

• El reglamento de tu universidad.

• Tus apuntes.

• La documentación interna.

• La versión más reciente de un procedimiento.

• Los archivos privados de un proyecto.

SuZuMaCyber Instructor

Nota para el docente: Pregunta al grupo: "¿Qué información necesitaría un asistente de IA para responder sobre nuestra propia universidad o proyecto?"

Sugerencia didáctica: pedir al estudiante que explique el concepto con sus propias palabras y relacionarlo con SuZuMa Tutor.

4. Tres caminos distintos

CONOCIMIENTO DEL MODELO

Lo aprendido durante su entrenamiento.

MEMORIA / ESTADO

Información guardada de la interacción.

RAG

Información recuperada desde fuentes externas antes de responder.

SuZuMaCyber Instructor

Nota para el docente: Aclara la diferencia conceptual. RAG permite incorporar información sin tener que reentrenar el modelo.

Sugerencia didáctica: pedir al estudiante que explique el concepto con sus propias palabras y relacionarlo con SuZuMa Tutor.

5. ¿Qué significa RAG?

Retrieval-Augmented Generation

1. Retrieval → recuperar información.

2. Augmented → agregarla al contexto.

3. Generation → generar una respuesta con ese contexto.

SuZuMaCyber Instructor

Nota para el docente: Usa el flujo como idea central de todo el módulo.

Sugerencia didáctica: pedir al estudiante que explique el concepto con sus propias palabras y relacionarlo con SuZuMa Tutor.

6. Arquitectura RAG

DOCUMENTOS

↓

Carga y limpieza

↓

Chunking

↓

Embeddings

↓

Base vectorial

↓

Búsqueda semántica

↓

Contexto relevante

↓

LLM

↓

Respuesta fundamentada

SuZuMaCyber Instructor

Nota para el docente: Explica que la calidad final depende tanto de la recuperación como del modelo generativo.

Sugerencia didáctica: pedir al estudiante que explique el concepto con sus propias palabras y relacionarlo con SuZuMa Tutor.

7. Paso 1 — Ingesta

Fuentes comunes:

• TXT

• Markdown

• PDF

• DOCX

• HTML

• Bases de datos

Objetivo: convertir diferentes formatos a texto procesable.

SuZuMaCyber Instructor

> *Nota para el docente: No hace falta implementar todos los formatos en el laboratorio. Empieza con TXT/Markdown para entender el proceso.*

> *Sugerencia didáctica: pedir al estudiante que explique el concepto con sus propias palabras y relacionarlo con SuZuMa Tutor.*

8. Paso 2 — Chunking

Un documento grande se divide en fragmentos pequeños.

Ejemplo:

Documento → 1,000 párrafos → fragmentos de 300–600 palabras.

Cada fragmento debe conservar suficiente contexto para ser útil.

SuZuMaCyber Instructor

> *Nota para el docente: Introduce la idea de tamaño de chunk y solapamiento. Evita presentarlo como una cifra universal.*

> *Sugerencia didáctica: pedir al estudiante que explique el concepto con sus propias palabras y relacionarlo con SuZuMa Tutor.*

9. ¿Por qué dividir el documento?

Porque recuperar un documento completo puede ser:

• Costoso.

• Poco preciso.

• Demasiado grande para el contexto.

• Difícil de relacionar con una pregunta concreta.

Queremos recuperar “lo necesario”.

SuZuMaCyber Instructor

> *Nota para el docente: Relaciona chunking con una biblioteca: no quieres traer todos los libros para responder una pregunta.*

> *Sugerencia didáctica: pedir al estudiante que explique el concepto con sus propias palabras y relacionarlo con SuZuMa Tutor.*

10. Overlap: conservar contexto

Fragmento A: …concepto → explicación → ejemplo

■

Fragmento B: explicación → ejemplo → siguiente idea…

El solapamiento ayuda a evitar cortes bruscos entre conceptos.

SuZuMaCyber Instructor

Nota para el docente: Explica que demasiado overlap puede aumentar almacenamiento y ruido. Debe probarse según el contenido.

Sugerencia didáctica: pedir al estudiante que explique el concepto con sus propias palabras y relacionarlo con SuZuMa Tutor.

11. Paso 3 — Embeddings

Un embedding representa texto como un vector numérico.

Textos con significado parecido tienden a quedar cerca en el espacio vectorial.

“Cómo solicitar una beca”

≈

“Requisitos para obtener apoyo estudiantil”

SuZuMaCyber Instructor

Nota para el docente: No es necesario enseñar la matemática interna para usar embeddings. Lo importante es comprender la representación semántica.

Sugerencia didáctica: pedir al estudiante que explique el concepto con sus propias palabras y relacionarlo con SuZuMa Tutor.

12. Búsqueda semántica

Pregunta:

“¿Qué requisitos necesito para solicitar una beca?”

La búsqueda no depende únicamente de coincidencias exactas.

Busca fragmentos relacionados por significado.

SuZuMaCyber Instructor

Nota para el docente: Contrasta mentalmente con una búsqueda por palabras clave.

Sugerencia didáctica: pedir al estudiante que explique el concepto con sus propias palabras y relacionarlo con SuZuMa Tutor.

13. Base vectorial

Guarda información como:

• Vector / embedding.

• Texto del fragmento.

• Metadatos.

Ejemplo de metadatos:

{ documento, página, sección, fecha }

SuZuMaCyber Instructor

Nota para el docente: Introduce los metadatos como elemento clave para trazabilidad y filtros.

Sugerencia didáctica: pedir al estudiante que explique el concepto con sus propias palabras y relacionarlo con SuZuMa Tutor.

14. Top-k: ¿cuántos fragmentos recuperar?

La consulta puede recuperar los fragmentos más relevantes.

TOP-K = número de resultados recuperados.

k pequeño → menos ruido.

k grande → más contexto, pero puede introducir información irrelevante.

SuZuMaCyber Instructor

Nota para el docente: No fijes un valor como regla universal. En el laboratorio se puede experimentar con k=3 y k=5.

Sugerencia didáctica: pedir al estudiante que explique el concepto con sus propias palabras y relacionarlo con SuZuMa Tutor.

15. Paso 4 — Construir el contexto

Pregunta del usuario

+

Fragmentos recuperados

+

Instrucciones del agente

↓

CONTEXTO PARA EL MODELO

SuZuMaCyber Instructor

Nota para el docente: Aquí conecta directamente con el módulo anterior de Tool Calling: recuperar información es otra capacidad que puede convertirse en herramienta.

Sugerencia didáctica: pedir al estudiante que explique el concepto con sus propias palabras y relacionarlo con SuZuMa Tutor.

16. Paso 5 — Generar con evidencia

Regla recomendada:

“Responde usando únicamente la información recuperada. Si la respuesta no está respaldada por los documentos, indícalo.”

Resultado esperado:

menos invención + mayor trazabilidad.

SuZuMaCyber Instructor

Nota para el docente: Habla de respuestas fundamentadas, no de “cero alucinaciones”. Ningún sistema garantiza cero errores.

Sugerencia didáctica: pedir al estudiante que explique el concepto con sus propias palabras y relacionarlo con SuZuMa Tutor.

17. Citas y trazabilidad

Una buena respuesta puede indicar:

Fuente: Reglamento_Alumno.pdf

Página: 12

Sección: Becas

Esto permite al usuario verificar la respuesta original.

SuZuMaCyber Instructor

Nota para el docente: Las citas dependen de que conservemos metadatos durante la ingesta y recuperación.

Sugerencia didáctica: pedir al estudiante que explique el concepto con sus propias palabras y relacionarlo con SuZuMa Tutor.

18. RAG vs fine-tuning

RAG

• Consulta información externa.

• Fácil de actualizar.

• Conserva fuentes.

• Útil para conocimiento documental.

Fine-tuning

• Ajusta el comportamiento/modelo.

• Puede ser útil para patrones de salida o tareas específicas.

• No sustituye automáticamente una base documental actualizada.

SuZuMaCyber Instructor

Nota para el docente: Evita presentar una técnica como reemplazo universal de la otra. Son herramientas para problemas diferentes.

Sugerencia didáctica: pedir al estudiante que explique el concepto con sus propias palabras y relacionarlo con SuZuMa Tutor.

19. Seguridad de documentos

Un RAG también puede filtrar información sensible.

Buenas prácticas:

• Controlar acceso a documentos.

• Validar fuentes.

• Separar datos por usuario/rol.

• No exponer documentos completos innecesariamente.

• Registrar consultas y errores cuando corresponda.

SuZuMaCyber Instructor

Nota para el docente: Lleva la conversación hacia seguridad desde el diseño, especialmente para proyectos universitarios.

Sugerencia didáctica: pedir al estudiante que explique el concepto con sus propias palabras y relacionarlo con SuZuMa Tutor.

20. Laboratorio — SuZuMa Tutor V4

OBJETIVO

Crear un tutor que responda preguntas usando un conjunto de documentos del curso.

Pipeline:

1. Cargar documentos.

2. Dividir en chunks.

3. Crear embeddings.

4. Guardar en almacén vectorial.

5. Recuperar top-k.

6. Construir contexto.

7. Generar respuesta.

SuZuMaCyber Instructor

Nota para el docente: Duración sugerida: 45–60 minutos. Empieza con documentos pequeños y controlados.

Sugerencia didáctica: pedir al estudiante que explique el concepto con sus propias palabras y relacionarlo con SuZuMa Tutor.

21. Diseño del laboratorio

Entrada:

“¿Qué debo entregar en el proyecto final?”

Retriever → recupera 3 fragmentos.

Contexto → fragmentos + reglas.

LLM → respuesta.

Salida → respuesta + fuentes.

SuZuMaCyber Instructor

Nota para el docente: Pide al estudiante que pruebe preguntas que sí están y que no están en los documentos.

Sugerencia didáctica: pedir al estudiante que explique el concepto con sus propias palabras y relacionarlo con SuZuMa Tutor.

22. Pruebas de calidad

Prueba 1: pregunta explícitamente documentada.

Prueba 2: pregunta con sinónimos.

Prueba 3: pregunta que combine dos secciones.

Prueba 4: información inexistente.

Prueba 5: documento con información contradictoria.

SuZuMaCyber Instructor

Nota para el docente: El objetivo es observar dónde falla el sistema: recuperación, contexto o generación.

Sugerencia didáctica: pedir al estudiante que explique el concepto con sus propias palabras y relacionarlo con SuZuMa Tutor.

23. Reto — SuZuMa Tutor V4

Entrega:

• Carpeta de documentos.

• Pipeline de ingesta.

• Chunking configurable.

• Embeddings.

• Recuperación top-k.

• Prompt de respuesta fundamentada.

• Fuentes/metadatos.

• Pruebas documentadas.

SuZuMaCyber Instructor

Nota para el docente: Pide una pequeña demostración donde el estudiante explique por qué se recuperó cada fragmento.

Sugerencia didáctica: pedir al estudiante que explique el concepto con sus propias palabras y relacionarlo con SuZuMa Tutor.

24. Cierre y siguiente paso

V1 → Conversación

V2 → Memoria

V3 → Herramientas

V4 → RAG

SIGUIENTE:

Agentes que planifican, usan herramientas y coordinan varios pasos.

SuZuMaCyber Instructor

> *Nota para el docente: Cierra mostrando la evolución del mismo proyecto. El siguiente módulo puede introducir planificación, orquestación y agentes multi-herramienta.*

> *Sugerencia didáctica: pedir al estudiante que explique el concepto con sus propias palabras y relacionarlo con SuZuMa Tutor.*

25. RAG no es una búsqueda simple

Una búsqueda tradicional suele depender de palabras o filtros.

RAG combina:

• recuperación por significado;

• selección de fragmentos relevantes;

• construcción de contexto;

• generación de una respuesta;

• y, cuando se diseña así, referencias a las fuentes.

SuZuMaCyber Instructor

> *Nota para el docente: Usa esta diapositiva para diferenciar RAG de una búsqueda convencional. El punto clave es que el sistema no solo encuentra información: recupera contexto que posteriormente se utiliza para generar una respuesta. Pide al grupo que explique qué etapa podría fallar si una respuesta resulta incorrecta.*

Cierre y evidencia sugerida

Pedir al estudiante una explicación breve del concepto central, una demostración práctica y una evidencia verificable (código, prueba, diagrama, resultado o registro).

Módulo 7 — Planificación y Orquestación

1. MÓDULO 7

Planificación y Orquestación de Agentes

SuZuMaCyber Instructor

Nota para el docente: De recuperar información a decidir qué hacer, en qué orden y con qué herramientas.

Sugerencia didáctica: pedir al estudiante que relacione el concepto con SuZuMa Tutor y explique qué parte del sistema interviene.

2. Objetivos del módulo

Al terminar podrás:

• Explicar qué es planificación en un agente.

• Descomponer objetivos en tareas.

• Diseñar flujos de ejecución.

• Coordinar herramientas y RAG.

• Manejar errores y reintentos.

• Construir un agente multi-paso.

SuZuMaCyber Instructor

Nota para el docente: Enfatiza que un agente no es solo un chatbot con muchas funciones: necesita un proceso para decidir y ejecutar.

Sugerencia didáctica: pedir al estudiante que relacione el concepto con SuZuMa Tutor y explique qué parte del sistema interviene.

3. ¿Qué cambia en un agente?

CHATBOT

Pregunta → Respuesta

AGENTE

Objetivo → Plan → Acción → Observación → Decisión → Nueva acción → Resultado

SuZuMaCyber Instructor

Nota para el docente: Pide al grupo un ejemplo cotidiano donde una tarea requiera varios pasos.

Sugerencia didáctica: pedir al estudiante que relacione el concepto con SuZuMa Tutor y explique qué parte del sistema interviene.

4. Un objetivo complejo

Ejemplo:

“Prepara una guía de estudio para el examen de Redes.”

Puede requerir:

1. Consultar documentos.

2. Identificar temas.

3. Organizar contenidos.

4. Crear preguntas.

5. Revisar cobertura.

6. Entregar resultado.

SuZuMaCyber Instructor

Nota para el docente: La clave es convertir un objetivo amplio en pasos verificables.

Sugerencia didáctica: pedir al estudiante que relacione el concepto con SuZuMa Tutor y explique qué parte del sistema interviene.

5. Descomposición de tareas

OBJETIVO

↓

TAREA 1

↓

TAREA 2

↓

TAREA 3

↓

VALIDACIÓN

↓

RESULTADO

Cada tarea debe tener una entrada, una acción y una salida esperada.

SuZuMaCyber Instructor

Nota para el docente: Relaciona esto con programación modular: funciones pequeñas con responsabilidades claras.

Sugerencia didáctica: pedir al estudiante que relacione el concepto con SuZuMa Tutor y explique qué parte del sistema interviene.

6. Planificación secuencial

Cuando una tarea depende de la anterior:

A → B → C → D

Ejemplo:

Buscar documentos → recuperar evidencia → redactar → validar → responder

SuZuMaCyber Instructor

Nota para el docente: Es el flujo más sencillo y el mejor punto de partida para estudiantes.

Sugerencia didáctica: pedir al estudiante que relacione el concepto con SuZuMa Tutor y explique qué parte del sistema interviene.

7. Planificación condicional

A veces el siguiente paso depende de una condición:

Buscar datos

↓

¿Hay resultados?

■■ Sí → analizar

■■ No → pedir aclaración / informar limitación

SuZuMaCyber Instructor

Nota para el docente: Introduce decisiones explícitas. El agente debe saber qué hacer cuando una herramienta no produce resultados.

Sugerencia didáctica: pedir al estudiante que relacione el concepto con SuZuMa Tutor y explique qué parte del sistema interviene.

8. Planificación con herramientas

AGENTE

↓

Decide

↓

Tool: buscar_documentos

↓

Observa resultado

↓

Decide

↓

Tool: generar_resumen

↓

Observa

↓

Respuesta

SuZuMaCyber Instructor

Nota para el docente: Conecta directamente con el Módulo 5: las herramientas son las capacidades que el agente puede ejecutar.

Sugerencia didáctica: pedir al estudiante que relacione el concepto con SuZuMa Tutor y explique qué parte del sistema interviene.

9. Planificación + RAG

Un agente puede combinar:

RAG → encontrar conocimiento.

Tools → ejecutar acciones.

Memoria → conservar estado.

LLM → interpretar y decidir.

La orquestación coordina todos estos componentes.

SuZuMaCyber Instructor

Nota para el docente: Presenta estos cuatro elementos como piezas complementarias de SuZuMa Tutor.

Sugerencia didáctica: pedir al estudiante que relacione el concepto con SuZuMa Tutor y explique qué parte del sistema interviene.

10. El bucle del agente

1. OBJETIVO

2. PLAN

3. ACCIÓN

4. OBSERVACIÓN

5. EVALUACIÓN

6. SIGUIENTE ACCIÓN

7. FINALIZACIÓN

SuZuMaCyber Instructor

Nota para el docente: Este ciclo puede repetirse varias veces. Hay que definir condiciones claras de salida.

Sugerencia didáctica: pedir al estudiante que relacione el concepto con SuZuMa Tutor y explique qué parte del sistema interviene.

11. Estado del agente

El agente necesita conocer:

• Objetivo actual.

• Paso actual.

• Resultados anteriores.

• Herramientas disponibles.

• Errores ocurridos.

• Condiciones de finalización.

SuZuMaCyber Instructor

Nota para el docente: Distingue estado temporal de memoria persistente. No todo lo que ocurre necesita guardarse permanentemente.

Sugerencia didáctica: pedir al estudiante que relacione el concepto con SuZuMa Tutor y explique qué parte del sistema interviene.

12. Plan estructurado

Ejemplo conceptual:

{

"objetivo": "crear guía",

"pasos": [

"recuperar documentos",

"organizar temas",

"crear preguntas",

"validar"

]

}

SuZuMaCyber Instructor

Nota para el docente: El formato estructurado facilita inspección, validación y depuración.

Sugerencia didáctica: pedir al estudiante que relacione el concepto con SuZuMa Tutor y explique qué parte del sistema interviene.

13. ¿Plan fijo o plan dinámico?

PLAN FIJO

El programador define el flujo.

PLAN DINÁMICO

El agente decide los siguientes pasos según el contexto.

HÍBRIDO

Reglas críticas fijas + decisiones flexibles.

SuZuMaCyber Instructor

> *Nota para el docente: Explica que para sistemas reales suele ser conveniente limitar la autonomía donde existen riesgos o requisitos estrictos.*

> *Sugerencia didáctica: pedir al estudiante que relacione el concepto con SuZuMa Tutor y explique qué parte del sistema interviene.*

14. Orquestador

El orquestador coordina:

• qué tarea ejecutar;

• qué herramienta utilizar;

• qué resultado aceptar;

• cuándo repetir;

• cuándo detenerse;

• cuándo pedir intervención humana.

SuZuMaCyber Instructor

> *Nota para el docente: El orquestador puede ser código tradicional, un flujo estructurado o una combinación con un modelo.*

> *Sugerencia didáctica: pedir al estudiante que relacione el concepto con SuZuMa Tutor y explique qué parte del sistema interviene.*

15. Errores y reintentos

Ejemplo:

API falla

↓

¿Reintentable?

■■ Sí → esperar → reintentar

■■ No → cambiar estrategia / informar

Nunca conviene repetir indefinidamente.

SuZuMaCyber Instructor

> *Nota para el docente: Introduce límite de reintentos y registro del error.*

> *Sugerencia didáctica: pedir al estudiante que relacione el concepto con SuZuMa Tutor y explique qué parte del sistema interviene.*

16. Guardrails del agente

Define límites:

• Herramientas permitidas.

• Parámetros válidos.

• Máximo de pasos.

• Tiempo máximo.

• Acciones que requieren aprobación.

• Datos que nunca deben exponerse.

SuZuMaCyber Instructor

Nota para el docente: La autonomía debe estar acompañada por controles.

Sugerencia didáctica: pedir al estudiante que relacione el concepto con SuZuMa Tutor y explique qué parte del sistema interviene.

17. Human-in-the-loop

Algunas acciones pueden requerir aprobación:

Agente prepara → Usuario revisa → Usuario aprueba → Agente ejecuta

Ejemplos:

• enviar correo;

• modificar registros;

• publicar contenido;

• realizar una acción irreversible.

SuZuMaCyber Instructor

Nota para el docente: Diferencia preparación de ejecución. El usuario conserva control en acciones sensibles.

Sugerencia didáctica: pedir al estudiante que relacione el concepto con SuZuMa Tutor y explique qué parte del sistema interviene.

18. Observabilidad

Para depurar un agente registra:

• objetivo;

• paso ejecutado;

• herramienta usada;

• parámetros relevantes;

• resultado;

• error;

• tiempo;

• decisión siguiente.

SuZuMaCyber Instructor

Nota para el docente: La trazabilidad permite saber por qué un agente llegó a una respuesta o falló.

Sugerencia didáctica: pedir al estudiante que relacione el concepto con SuZuMa Tutor y explique qué parte del sistema interviene.

19. Laboratorio — SuZuMa Tutor V5

Objetivo:

Crear un agente que reciba una tarea y construya un plan.

Ejemplo:

“Prepara una guía sobre redes usando mis documentos.”

Debe:

1. Planificar.

2. Consultar RAG.

3. Organizar información.

4. Validar cobertura.

5. Generar resultado.

SuZuMaCyber Instructor

Nota para el docente: Duración sugerida: 50–60 minutos.

Sugerencia didáctica: pedir al estudiante que relacione el concepto con SuZuMa Tutor y explique qué parte del sistema interviene.

20. Arquitectura del laboratorio

USUARIO

↓

AGENTE / ORQUESTADOR

↓

PLAN

■■ RAG

■■ TOOL

■■ MEMORIA / ESTADO

■■ VALIDACIÓN

↓

RESULTADO

SuZuMaCyber Instructor

Nota para el docente: Pide a los estudiantes dibujar primero la arquitectura antes de programarla.

Sugerencia didáctica: pedir al estudiante que relacione el concepto con SuZuMa Tutor y explique qué parte del sistema interviene.

21. Control de ejecución

Reglas del laboratorio:

• Máximo 6 pasos.

• Máximo 2 reintentos por herramienta.

• No ejecutar herramientas desconocidas.

• Si falta información, detenerse y explicarlo.

• Registrar cada paso.

SuZuMaCyber Instructor

Nota para el docente: Estas reglas convierten un prototipo abierto en un sistema que puede probarse.

Sugerencia didáctica: pedir al estudiante que relacione el concepto con SuZuMa Tutor y explique qué parte del sistema interviene.

22. Pruebas del agente

Caso A: tarea sencilla.

Caso B: tarea con varias fuentes.

Caso C: herramienta con error.

Caso D: información inexistente.

Caso E: tarea ambigua.

Caso F: límite de pasos alcanzado.

SuZuMaCyber Instructor

Nota para el docente: El estudiante debe documentar qué decidió el agente y por qué.

Sugerencia didáctica: pedir al estudiante que relacione el concepto con SuZuMa Tutor y explique qué parte del sistema interviene.

23. Reto — SuZuMa Tutor V5

Entrega:

• Orquestador.

• Plan estructurado.

• Integración RAG.

• Al menos una herramienta.

• Estado de ejecución.

• Manejo de errores.

• Límite de pasos.

• Registro de ejecución.

• 6 casos de prueba.

SuZuMaCyber Instructor

Nota para el docente: Pide una demostración donde el estudiante pueda señalar cada paso del agente.

Sugerencia didáctica: pedir al estudiante que relacione el concepto con SuZuMa Tutor y explique qué parte del sistema interviene.

24. Cierre — hacia agentes más autónomos

V1 → Conversación

V2 → Memoria

V3 → Herramientas

V4 → RAG

V5 → Planificación y orquestación

SIGUIENTE:

Arquitecturas multiagente y coordinación de agentes.

SuZuMaCyber Instructor

Nota para el docente: La meta no es crear un agente que haga todo sin control, sino un sistema capaz de coordinar tareas de forma verificable.

Sugerencia didáctica: pedir al estudiante que relacione el concepto con SuZuMa Tutor y explique qué parte del sistema interviene.

25. Checklist del agente antes de ejecutarlo

■ ¿El objetivo está claramente definido?

■ ¿Cada paso tiene una salida verificable?

■ ¿Las herramientas tienen límites?

■ ¿Existe una condición de finalización?

■ ¿Hay manejo de errores y reintentos?

■ ¿Las acciones sensibles requieren aprobación?

■ ¿Podemos reconstruir qué hizo el agente?

SuZuMaCyber Instructor

> *Nota para el docente: Utiliza este checklist como cierre práctico. Antes de permitir que un agente ejecute acciones, el estudiante debe poder explicar su objetivo, sus límites, sus condiciones de salida y cómo se recupera ante errores.*

Cierre y evidencia sugerida

Pedir al estudiante una explicación breve del concepto central, una demostración práctica y una evidencia verificable (código, prueba, diagrama, resultado o registro).

Módulo 8 — Arquitecturas Multiagente

1. MÓDULO 8

Arquitecturas Multiagente

SuZuMaCyber Instructor

Nota para el docente: De un agente que coordina herramientas a varios agentes especializados que colaboran.

Sugerencia didáctica: pedir al estudiante que justifique la arquitectura elegida y qué problema resuelve.

2. Objetivos del módulo

Al terminar podrás:

• Explicar cuándo tiene sentido usar varios agentes.

• Identificar roles especializados.

• Diseñar comunicación entre agentes.

• Comparar arquitecturas de coordinación.

• Definir responsabilidades y límites.

• Construir un prototipo multiagente.

SuZuMaCyber Instructor

Nota para el docente: Aclara que multiagente no significa automáticamente mejor. La arquitectura debe justificarse por la complejidad del problema.

Sugerencia didáctica: pedir al estudiante que justifique la arquitectura elegida y qué problema resuelve.

3. ¿Por qué varios agentes?

Un solo agente puede encargarse de todo.

Pero una tarea compleja puede dividirse en especialistas:

• Investigador.

• Analista.

• Generador.

• Revisor.

• Coordinador.

SuZuMaCyber Instructor

Nota para el docente: Pregunta: “¿Qué ventajas tendría separar investigación, redacción y revisión?”

Sugerencia didáctica: pedir al estudiante que justifique la arquitectura elegida y qué problema resuelve.

4. Agente especialista

Un especialista tiene:

• Objetivo concreto.

• Herramientas específicas.

• Instrucciones propias.

• Entrada definida.

• Salida definida.

• Límites de actuación.

SuZuMaCyber Instructor

Nota para el docente: La especialización facilita probar y depurar cada componente.

Sugerencia didáctica: pedir al estudiante que justifique la arquitectura elegida y qué problema resuelve.

5. Ejemplo — Equipo de estudio

COORDINADOR

■■ Investigador → recupera fuentes

■■ Analista → organiza conceptos

■■ Generador → crea material

■■ Revisor → verifica cobertura

Resultado → guía de estudio

SuZuMaCyber Instructor

Nota para el docente: Este será el modelo conceptual del laboratorio.

Sugerencia didáctica: pedir al estudiante que justifique la arquitectura elegida y qué problema resuelve.

6. Multiagente vs. multi-herramienta

MULTI-HERRAMIENTA

Un agente decide entre varias herramientas.

MULTIAGENTE

Varios agentes tienen responsabilidades diferenciadas y colaboran.

Pueden coexistir en la misma arquitectura.

SuZuMaCyber Instructor

Nota para el docente: Relaciona con módulos 5 y 7.

Sugerencia didáctica: pedir al estudiante que justifique la arquitectura elegida y qué problema resuelve.

7. Arquitectura 1 — Secuencial

Agente A

↓

Agente B

↓

Agente C

↓

Resultado

Útil cuando cada etapa depende de la anterior.

SuZuMaCyber Instructor

Nota para el docente: Ejemplo: investigar → analizar → redactar.

Sugerencia didáctica: pedir al estudiante que justifique la arquitectura elegida y qué problema resuelve.

8. Arquitectura 2 — Coordinador

COORDINADOR

/ | \

A B C

\ | /

RESULTADO

Un agente central distribuye tareas.

SuZuMaCyber Instructor

Nota para el docente: El coordinador decide quién trabaja y combina resultados.

Sugerencia didáctica: pedir al estudiante que justifique la arquitectura elegida y qué problema resuelve.

9. Arquitectura 3 — Jerárquica

COORDINADOR PRINCIPAL

↓

SUBCOORDINADOR

■■ Especialista A

■■ Especialista B

■■ Especialista C

Útil para problemas con varios niveles de descomposición.

SuZuMaCyber Instructor

Nota para el docente: Introduce jerarquías sin asumir que siempre sean necesarias.

Sugerencia didáctica: pedir al estudiante que justifique la arquitectura elegida y qué problema resuelve.

10. Arquitectura 4 — Debate / revisión

AGENTE A → propuesta

AGENTE B → crítica

AGENTE A → mejora

REVISOR → decisión

Puede aumentar la calidad, pero también aumenta costo y complejidad.

SuZuMaCyber Instructor

Nota para el docente: Presenta el patrón como una técnica de revisión, no como garantía de calidad.

Sugerencia didáctica: pedir al estudiante que justifique la arquitectura elegida y qué problema resuelve.

11. Contrato entre agentes

Cada agente debe definir:

INPUT

¿Qué recibe?

OUTPUT

¿Qué entrega?

REGLAS

¿Qué puede y no puede hacer?

CRITERIO DE ÉXITO

¿Cómo sabemos que terminó correctamente?

SuZuMaCyber Instructor

Nota para el docente: Un contrato claro evita que un agente dependa de supuestos ocultos.

Sugerencia didáctica: pedir al estudiante que justifique la arquitectura elegida y qué problema resuelve.

12. Formato de mensajes

Ejemplo conceptual:

{

"task": "analizar_tema",

"context": "...",

"constraints": ["usar fuentes"],

"expected_output": "lista estructurada"

}

SuZuMaCyber Instructor

Nota para el docente: El mensaje debe ser estructurado cuando la coordinación lo requiera. No todo tiene que ser texto libre.

Sugerencia didáctica: pedir al estudiante que justifique la arquitectura elegida y qué problema resuelve.

13. Estado compartido

Los agentes pueden necesitar compartir:

• objetivo global;

• tareas completadas;

• evidencias;

• decisiones;

• errores;

• resultados parciales.

El estado compartido debe ser mínimo y controlado.

SuZuMaCyber Instructor

Nota para el docente: Diferencia estado compartido de una memoria global ilimitada.

Sugerencia didáctica: pedir al estudiante que justifique la arquitectura elegida y qué problema resuelve.

14. Memoria en multiagente

Opciones:

• Memoria local por agente.

• Estado compartido.

• Memoria documental mediante RAG.

• Base de datos común.

Elegir según necesidad.

SuZuMaCyber Instructor

Nota para el docente: Pregunta qué información debería ser privada de cada agente y cuál debe compartirse.

Sugerencia didáctica: pedir al estudiante que justifique la arquitectura elegida y qué problema resuelve.

15. Ruteo de tareas

El coordinador puede preguntar:

¿Qué tipo de tarea es?

↓

¿Investigación? → Investigador

¿Cálculo? → Analista

¿Redacción? → Generador

¿Validación? → Revisor

SuZuMaCyber Instructor

Nota para el docente: El ruteo convierte el coordinador en un sistema de distribución de trabajo.

Sugerencia didáctica: pedir al estudiante que justifique la arquitectura elegida y qué problema resuelve.

16. Dependencias

No todas las tareas pueden ejecutarse al mismo tiempo.

A → B

A → C

B + C → D

Esto forma un grafo de dependencias.

SuZuMaCyber Instructor

Nota para el docente: Introduce DAG de forma intuitiva: algunas tareas pueden ser paralelas y otras dependen de resultados previos.

Sugerencia didáctica: pedir al estudiante que justifique la arquitectura elegida y qué problema resuelve.

17. Paralelismo

Si dos tareas son independientes:

■→ Investigador A ■■

Objetivo ■ ■→ Integrador

■→ Investigador B ■■

Puede reducir tiempo, pero requiere coordinación.

SuZuMaCyber Instructor

Nota para el docente: Aclarar que paralelismo también introduce sincronización y manejo de fallos.

Sugerencia didáctica: pedir al estudiante que justifique la arquitectura elegida y qué problema resuelve.

18. Errores entre agentes

Si un agente falla:

1. Registrar error.

2. Determinar si es recuperable.

3. Reintentar o cambiar estrategia.

4. Informar al coordinador.

5. Detener si se supera el límite.

SuZuMaCyber Instructor

Nota para el docente: Un error local no debería convertirse automáticamente en un ciclo infinito.

Sugerencia didáctica: pedir al estudiante que justifique la arquitectura elegida y qué problema resuelve.

19. Seguridad multiagente

Cada agente debe tener:

• Mínimos permisos necesarios.

• Herramientas explícitamente permitidas.

• Validación de entradas.

• Límites de ejecución.

• Protección de información sensible.

• Registro de acciones.

SuZuMaCyber Instructor

Nota para el docente: La superficie de ataque crece cuando aumentan los agentes y herramientas.

Sugerencia didáctica: pedir al estudiante que justifique la arquitectura elegida y qué problema resuelve.

20. Laboratorio — SuZuMa Academy Team

Construir un equipo de agentes para crear una guía de estudio.

COORDINADOR

→ INVESTIGADOR

→ ANALISTA

→ GENERADOR

→ REVISOR

Entrada: tema académico.

Salida: guía revisada con fuentes.

SuZuMaCyber Instructor

Nota para el docente: Duración sugerida: 60 minutos.

Sugerencia didáctica: pedir al estudiante que justifique la arquitectura elegida y qué problema resuelve.

21. Flujo del laboratorio

1. Coordinador recibe objetivo.

2. Divide el trabajo.

3. Investigador consulta RAG.

4. Analista organiza hallazgos.

5. Generador crea contenido.

6. Revisor detecta problemas.

7. Coordinador decide si entrega o solicita mejora.

SuZuMaCyber Instructor

Nota para el docente: Pide que cada estudiante pueda señalar dónde ocurre cada transición.

Sugerencia didáctica: pedir al estudiante que justifique la arquitectura elegida y qué problema resuelve.

22. Pruebas multiagente

Caso A: tarea sencilla.

Caso B: información insuficiente.

Caso C: un agente falla.

Caso D: resultados contradictorios.

Caso E: tarea que no necesita varios agentes.

Caso F: límite de iteraciones.

SuZuMaCyber Instructor

Nota para el docente: El caso E es importante: obliga a justificar cuándo NO usar multiagente.

Sugerencia didáctica: pedir al estudiante que justifique la arquitectura elegida y qué problema resuelve.

23. Reto — SuZuMa Tutor V6

Entrega:

• Arquitectura multiagente.

• Coordinador.

• 3 especialistas mínimo.

• Contratos de entrada/salida.

• Estado de ejecución.

• Manejo de errores.

• Límite de iteraciones.

• Registro de mensajes.

• Casos de prueba.

SuZuMaCyber Instructor

Nota para el docente: Pide una demostración del flujo completo y una explicación de por qué cada agente existe.

Sugerencia didáctica: pedir al estudiante que justifique la arquitectura elegida y qué problema resuelve.

24. Checklist de diseño

Antes de crear otro agente:

■ ¿Existe una responsabilidad claramente distinta?

■ ¿Tiene herramientas o contexto diferente?

■ ¿Su salida puede validarse?

■ ¿Reduce complejidad o la aumenta?

■ ¿Tiene límites claros?

■ ¿Podría resolverse con un solo agente?

SuZuMaCyber Instructor

> *Nota para el docente: Esta lista ayuda a evitar arquitecturas innecesariamente complejas.*

> *Sugerencia didáctica: pedir al estudiante que justifique la arquitectura elegida y qué problema resuelve.*

25. Cierre — evolución del sistema

V1 → Conversación

V2 → Memoria

V3 → Herramientas

V4 → RAG

V5 → Orquestación

V6 → Multiagente

SIGUIENTE:

Evaluación, observabilidad y despliegue de agentes.

SuZuMaCyber Instructor

> *Nota para el docente: El siguiente paso debe llevar el prototipo hacia un sistema medible, controlable y preparado para ejecutarse fuera del entorno de desarrollo.*

> *Sugerencia didáctica: pedir al estudiante que justifique la arquitectura elegida y qué problema resuelve.*

Cierre y evidencia sugerida

Pedir al estudiante una explicación breve del concepto central, una demostración práctica y una evidencia verificable (código, prueba, diagrama, resultado o registro).

Módulo 9 — Evaluación, Observabilidad y Despliegue

1. MÓDULO 9

Evaluación, Observabilidad y Despliegue

SuZuMaCyber Instructor

Nota para el docente: De un prototipo que funciona a un agente que podemos medir, depurar y operar.

Sugerencia didáctica: relacionar el concepto con el SuZuMa Tutor construido durante los módulos anteriores.

2. Objetivos del módulo

Al terminar podrás:

• Definir criterios de calidad para un agente.

• Crear casos de prueba reproducibles.

• Medir recuperación y respuestas.

• Observar ejecuciones y errores.

• Diseñar registros útiles.

• Preparar un despliegue básico.

SuZuMaCyber Instructor

Nota para el docente: Enfatiza que “funciona en mi computadora” no equivale a “sistema listo para usuarios”.

Sugerencia didáctica: relacionar el concepto con el SuZuMa Tutor construido durante los módulos anteriores.

3. ¿Cómo sabemos si funciona?

No basta con preguntar: “¿respondió?”

Debemos evaluar:

• Exactitud.

• Fundamentación.

• Uso correcto de herramientas.

• Cumplimiento de instrucciones.

• Tiempo de respuesta.

• Costo.

• Seguridad.

• Robustez ante errores.

SuZuMaCyber Instructor

Nota para el docente: La evaluación debe diseñarse antes de cambiar continuamente el agente.

Sugerencia didáctica: relacionar el concepto con el SuZuMa Tutor construido durante los módulos anteriores.

4. Dataset de evaluación

Construye un conjunto de preguntas o tareas con:

INPUT → comportamiento esperado → criterios de evaluación

Incluye casos normales, ambiguos, difíciles y fuera de alcance.

SuZuMaCyber Instructor

Nota para el docente: Sugiere comenzar con 20–30 casos representativos para un proyecto académico.

Sugerencia didáctica: relacionar el concepto con el SuZuMa Tutor construido durante los módulos anteriores.

5. Golden set

Un “golden set” es un conjunto estable de casos de referencia.

Ejemplo:

• 10 preguntas documentales.

• 5 preguntas de herramientas.

• 5 casos ambiguos.

• 5 casos sin respuesta conocida.

SuZuMaCyber Instructor

Nota para el docente: El conjunto debe mantenerse estable para comparar versiones del agente.

Sugerencia didáctica: relacionar el concepto con el SuZuMa Tutor construido durante los módulos anteriores.

6. Evaluar RAG

Separar dos preguntas:

1. ¿Recuperó información relevante?

2. ¿La respuesta utilizó correctamente esa información?

Un buen generador no puede compensar siempre una mala recuperación.

SuZuMaCyber Instructor

Nota para el docente: Relaciona directamente con el Módulo 6.

Sugerencia didáctica: relacionar el concepto con el SuZuMa Tutor construido durante los módulos anteriores.

7. Evaluar herramientas

Verifica:

• ¿Eligió la herramienta correcta?

• ¿Usó parámetros válidos?

• ¿Interpretó correctamente el resultado?

• ¿Manejó errores?

• ¿Evitó herramientas innecesarias?

SuZuMaCyber Instructor

Nota para el docente: Esto permite distinguir un problema del modelo de un problema de integración.

Sugerencia didáctica: relacionar el concepto con el SuZuMa Tutor construido durante los módulos anteriores.

8. Evaluar planificación

Observa:

• ¿Descompuso correctamente el objetivo?

• ¿Respetó dependencias?

• ¿Terminó cuando debía?

• ¿Repitió pasos innecesariamente?

• ¿Pidió aclaración cuando faltaba información?

SuZuMaCyber Instructor

Nota para el docente: No evalúes solo el resultado final. El camino también importa en un agente.

Sugerencia didáctica: relacionar el concepto con el SuZuMa Tutor construido durante los módulos anteriores.

9. Evaluar multiagente

Para cada agente:

• responsabilidad clara;

• entrada válida;

• salida útil;

• coordinación correcta;

• errores controlados.

Y para el sistema:

¿La colaboración aporta valor real?

SuZuMaCyber Instructor

Nota para el docente: Recuerda el checklist del módulo 8: más agentes también significa más complejidad.

Sugerencia didáctica: relacionar el concepto con el SuZuMa Tutor construido durante los módulos anteriores.

10. Métricas técnicas

Ejemplos:

Latencia → cuánto tarda.

Tasa de éxito → tareas completadas.

Errores → fallos por ejecución.

Reintentos → frecuencia de recuperación.

Costo → recursos consumidos.

Disponibilidad → tiempo operativo.

SuZuMaCyber Instructor

Nota para el docente: Aclara que una métrica aislada no describe toda la calidad.

Sugerencia didáctica: relacionar el concepto con el SuZuMa Tutor construido durante los módulos anteriores.

11. Métricas de calidad

Ejemplos:

• Respuesta correcta.

• Respuesta fundamentada.

• Cobertura de la pregunta.

• Relevancia de fuentes.

• Cumplimiento de formato.

• Satisfacción del usuario.

SuZuMaCyber Instructor

Nota para el docente: Combina métricas automáticas con revisión humana cuando sea necesario.

Sugerencia didáctica: relacionar el concepto con el SuZuMa Tutor construido durante los módulos anteriores.

12. Observabilidad

La observabilidad busca responder:

¿Qué ocurrió?

¿Por qué ocurrió?

¿Dónde falló?

¿Cuánto tardó?

¿Qué decisión tomó el agente?

SuZuMaCyber Instructor

Nota para el docente: Un sistema observable permite investigar una ejecución sin depender de reproducirla manualmente.

Sugerencia didáctica: relacionar el concepto con el SuZuMa Tutor construido durante los módulos anteriores.

13. Traza de una ejecución

EJECUCIÓN

■■ Entrada

■■ Plan

■■ Tool call

■■ Resultado

■■ Recuperación RAG

■■ Generación

■■ Validación

■■ Salida

Cada etapa puede registrar tiempo y estado.

SuZuMaCyber Instructor

Nota para el docente: Presenta la traza como una “historia técnica” de la ejecución.

Sugerencia didáctica: relacionar el concepto con el SuZuMa Tutor construido durante los módulos anteriores.

14. Logs útiles

Un log útil puede incluir:

• timestamp;

• identificador de ejecución;

• etapa;

• herramienta/agente;

• estado;

• duración;

• error resumido;

• resultado relevante.

SuZuMaCyber Instructor

Nota para el docente: Evita guardar datos sensibles innecesariamente. El logging también requiere diseño de privacidad.

Sugerencia didáctica: relacionar el concepto con el SuZuMa Tutor construido durante los módulos anteriores.

15. Errores y clasificación

Clasifica errores por origen:

INPUT → entrada inválida.

MODEL → decisión o generación incorrecta.

TOOL → integración/API.

RAG → recuperación/contexto.

ORCHESTRATION → flujo.

INFRA → red, servidor, recursos.

SuZuMaCyber Instructor

Nota para el docente: La clasificación ayuda a dirigir la corrección al componente adecuado.

Sugerencia didáctica: relacionar el concepto con el SuZuMa Tutor construido durante los módulos anteriores.

16. Seguridad antes del despliegue

Checklist:

• Secretos fuera del código.

• Autenticación.

• Autorización por herramienta.

• Validación de entradas.

• Límites de uso.

• Protección de datos.

• Logs sin información sensible.

• Dependencias actualizadas.

SuZuMaCyber Instructor

Nota para el docente: Haz énfasis en que los agentes pueden ejecutar acciones: los permisos deben ser mínimos.

Sugerencia didáctica: relacionar el concepto con el SuZuMa Tutor construido durante los módulos anteriores.

17. Del prototipo a producción

DESARROLLO

↓

PRUEBAS

↓

STAGING

↓

VALIDACIÓN

↓

PRODUCCIÓN

↓

MONITOREO

↓

MEJORA

SuZuMaCyber Instructor

Nota para el docente: Explica que desplegar no es el final: inicia el ciclo de operación y mejora.

Sugerencia didáctica: relacionar el concepto con el SuZuMa Tutor construido durante los módulos anteriores.

18. Configuración por entorno

Separar:

DESARROLLO

Datos de prueba.

STAGING

Pruebas integrales.

PRODUCCIÓN

Datos y servicios reales.

Configuración y secretos deben manejarse por entorno.

SuZuMaCyber Instructor

Nota para el docente: Evita que los estudiantes conecten accidentalmente un prototipo a servicios reales.

Sugerencia didáctica: relacionar el concepto con el SuZuMa Tutor construido durante los módulos anteriores.

19. Versionado del agente

Versiona al menos:

• código;

• prompts/instrucciones;

• herramientas;

• esquema de datos;

• configuración;

• dataset de evaluación.

Así puedes comparar V1, V2, V3…

SuZuMaCyber Instructor

Nota para el docente: Un cambio de prompt puede modificar el comportamiento tanto como un cambio de código.

Sugerencia didáctica: relacionar el concepto con el SuZuMa Tutor construido durante los módulos anteriores.

20. Laboratorio — SuZuMa Tutor V7

Tomar el agente multiagente anterior y convertirlo en un sistema evaluable.

Debe incluir:

1. Golden set.

2. Métricas.

3. Trazas.

4. Logs.

5. Clasificación de errores.

6. Configuración por entorno.

7. Checklist de seguridad.

SuZuMaCyber Instructor

Nota para el docente: Duración sugerida: 60 minutos.

Sugerencia didáctica: relacionar el concepto con el SuZuMa Tutor construido durante los módulos anteriores.

21. Diseño de pruebas

Crear al menos 10 pruebas:

• 3 normales.

• 2 ambiguas.

• 2 fuera de alcance.

• 1 fallo de herramienta.

• 1 fallo de recuperación.

• 1 caso de seguridad.

Registrar resultado esperado y resultado obtenido.

SuZuMaCyber Instructor

Nota para el docente: Pide que cada prueba sea reproducible.

Sugerencia didáctica: relacionar el concepto con el SuZuMa Tutor construido durante los módulos anteriores.

22. Dashboard conceptual

Un panel mínimo puede mostrar:

Ejecuciones → 1,250

Éxito → 91%

Errores → 9%

Latencia media → …

Reintentos → …

RAG sin evidencia → …

La visualización debe ayudar a detectar problemas, no solo decorar.

SuZuMaCyber Instructor

Nota para el docente: Los valores son ilustrativos. No presentarlos como datos reales.

Sugerencia didáctica: relacionar el concepto con el SuZuMa Tutor construido durante los módulos anteriores.

23. Reto final — SuZuMa Tutor V7

Entrega:

• Agente funcional.

• Evaluación reproducible.

• Golden set.

• Métricas.

• Trazas y logs.

• Manejo de errores.

• Seguridad básica.

• Configuración por entorno.

• Documento de resultados y mejoras propuestas.

SuZuMaCyber Instructor

Nota para el docente: Este reto integra los módulos anteriores en un sistema demostrable.

Sugerencia didáctica: relacionar el concepto con el SuZuMa Tutor construido durante los módulos anteriores.

24. Checklist final del curso

■ Conversación

■ Memoria

■ Herramientas

■ RAG

■ Planificación

■ Multiagente

■ Evaluación

■ Observabilidad

■ Seguridad

■ Despliegue

OBJETIVO: construir agentes que puedan entenderse, probarse y mejorarse.

SuZuMaCyber Instructor

Nota para el docente: Cierra el curso técnico resaltando que la ingeniería de agentes es un ciclo continuo: diseñar → probar → observar → mejorar.

Sugerencia didáctica: relacionar el concepto con el SuZuMa Tutor construido durante los módulos anteriores.

25. Cierre — del experimento al sistema

SuZuMa Tutor

V1 Conversación

V2 Memoria

V3 Herramientas

V4 RAG

V5 Orquestación

V6 Multiagente

V7 Evaluación y operación

Siguiente paso: proyecto integrador.

SuZuMaCyber Instructor

> *Nota para el docente: Invita al estudiante a integrar todo lo aprendido en un proyecto final con una necesidad real y criterios de éxito definidos.*

> *Sugerencia didáctica: relacionar el concepto con el SuZuMa Tutor construido durante los módulos anteriores.*

Cierre y evidencia sugerida

Pedir al estudiante una explicación breve del concepto central, una demostración práctica y una evidencia verificable (código, prueba, diagrama, resultado o registro).

Proyecto Integrador — Construye tu propio agente de IA

1. PROYECTO INTEGRADOR

Construye tu propio agente de IA

SuZuMaCyber Instructor

Nota para el docente: Proyecto final de SuZuMa Academy
De la idea → al prototipo → a un agente funcional, evaluable y demostrable.

Sugerencia didáctica: pedir al estudiante que conecte este punto con los módulos 1–9.

2. El reto

Construir un agente de IA que resuelva una necesidad real o académica.

El proyecto deberá integrar, según corresponda:

• conversación;

• memoria/estado;

• herramientas;

• RAG;

• planificación;

• multiagente;

• evaluación;

• observabilidad;

• seguridad.

SuZuMaCyber Instructor

Nota para el docente: Presenta el proyecto como integración de conocimientos, no como una carrera por agregar componentes.

Sugerencia didáctica: pedir al estudiante que conecte este punto con los módulos 1–9.

3. Pregunta guía

¿Qué problema concreto puede resolver tu agente?

Debe existir un usuario definido, una necesidad clara y un resultado verificable.

Ejemplos:

• tutor académico;

• asistente documental;

• soporte técnico;

• analizador de información;

• asistente para un proceso universitario.

SuZuMaCyber Instructor

Nota para el docente: Pide que el alumno describa el problema en una sola frase antes de escribir código.

Sugerencia didáctica: pedir al estudiante que conecte este punto con los módulos 1–9.

4. Regla principal

NO empezar por la tecnología.

Primero:

PROBLEMA → USUARIO → OBJETIVO → CRITERIOS DE ÉXITO

Después:

ARQUITECTURA → COMPONENTES → IMPLEMENTACIÓN

SuZuMaCyber Instructor

> *Nota para el docente: Evita que el proyecto se convierta en una colección de APIs sin propósito.*
>
> *Sugerencia didáctica: pedir al estudiante que conecte este punto con los módulos 1–9.*

5. Paso 1 — Definir al usuario

Documento inicial:

Usuario objetivo:

¿Qué persona utilizará el agente?

Contexto:

¿En qué situación lo utilizará?

Necesidad:

¿Qué problema intenta resolver?

Resultado:

¿Qué espera obtener?

SuZuMaCyber Instructor

> *Nota para el docente: Un proyecto puede tener varios usuarios, pero debe comenzar con uno claramente definido.*
>
> *Sugerencia didáctica: pedir al estudiante que conecte este punto con los módulos 1–9.*

6. Paso 2 — Definir alcance

INCLUIR

¿Qué hará el agente?

NO INCLUIR

¿Qué queda fuera?

RESTRICCIONES

¿Qué límites tendrá?

CRITERIOS DE ÉXITO

¿Cómo sabremos que funciona?

SuZuMaCyber Instructor

> *Nota para el docente: El alcance evita intentar construir un sistema demasiado grande para el tiempo disponible.*
>
> *Sugerencia didáctica: pedir al estudiante que conecte este punto con los módulos 1–9.*

7. Paso 3 — Historias de usuario

Formato:

“Como [usuario], quiero [acción], para [beneficio].”

Ejemplo:

“Como estudiante, quiero consultar mis apuntes mediante lenguaje natural, para preparar un examen.”

SuZuMaCyber Instructor

Nota para el docente: Pide al alumno entre 3 y 5 historias de usuario.

Sugerencia didáctica: pedir al estudiante que conecte este punto con los módulos 1–9.

8. Paso 4 — Arquitectura

Diseñar antes de programar:

USUARIO

↓

AGENTE

■■ Memoria / Estado

■■ Herramientas

■■ RAG

■■ Planificador

■■ Agentes especialistas

↓

RESPUESTA

SuZuMaCyber Instructor

Nota para el docente: No todos los proyectos necesitan todos los componentes.

Sugerencia didáctica: pedir al estudiante que conecte este punto con los módulos 1–9.

9. Arquitectura mínima viable

Comenzar con:

Entrada → Agente → Respuesta

Después agregar solo lo necesario:

+ herramienta

+ memoria

+ RAG

+ planificación

+ especialistas

SuZuMaCyber Instructor

Nota para el docente: La versión mínima permite tener algo funcional antes de aumentar la complejidad.

Sugerencia didáctica: pedir al estudiante que conecte este punto con los módulos 1–9.

10. Paso 5 — Diseñar herramientas

Para cada herramienta documentar:

• nombre;

• propósito;

• parámetros;

• salida;

• errores;

• permisos.

Pregunta clave:

¿Esta acción realmente necesita una herramienta?

SuZuMaCyber Instructor

Nota para el docente: Recupera el concepto de Tool Calling del módulo 5.

Sugerencia didáctica: pedir al estudiante que conecte este punto con los módulos 1–9.

11. Paso 6 — Diseñar conocimiento

Si el agente necesita documentos:

Documentos → chunks → embeddings → almacenamiento → recuperación → contexto → respuesta

Definir también fuentes y metadatos.

SuZuMaCyber Instructor

Nota para el docente: Relaciona con RAG. El alumno debe saber qué documentos son confiables para su proyecto.

Sugerencia didáctica: pedir al estudiante que conecte este punto con los módulos 1–9.

12. Paso 7 — Memoria y estado

Definir qué debe conservar:

ESTADO

Información necesaria durante la ejecución.

MEMORIA

Información que debe persistir entre interacciones.

NO GUARDAR

Datos innecesarios o sensibles.

SuZuMaCyber Instructor

Nota para el docente: Pedir justificar cada dato persistente.

Sugerencia didáctica: pedir al estudiante que conecte este punto con los módulos 1–9.

13. Paso 8 — Planificación

Si el objetivo tiene varios pasos:

Objetivo → plan → acción → observación → siguiente acción → validación

Definir:

• máximo de pasos;

• reintentos;

• condiciones de salida.

SuZuMaCyber Instructor

Nota para el docente: El plan debe ser observable y limitado.

Sugerencia didáctica: pedir al estudiante que conecte este punto con los módulos 1–9.

14. Paso 9 — Multiagente

Solo usarlo si aporta una separación clara.

Ejemplo:

COORDINADOR

■■ INVESTIGADOR

■■ ANALISTA

■■ GENERADOR

■■ REVISOR

Cada agente necesita una responsabilidad concreta.

SuZuMaCyber Instructor

Nota para el docente: La arquitectura multiagente es opcional si el problema se resuelve mejor con un solo agente.

Sugerencia didáctica: pedir al estudiante que conecte este punto con los módulos 1–9.

15. Paso 10 — Seguridad

Antes de probar:

■ Validar entradas.

■ Limitar herramientas.

■ Usar mínimos permisos.

■ Proteger secretos.

■ Evitar exponer información sensible.

■ Definir acciones que requieren aprobación.

■ Registrar errores sin datos innecesarios.

SuZuMaCyber Instructor

Nota para el docente: La seguridad forma parte del diseño, no es una etapa posterior.

Sugerencia didáctica: pedir al estudiante que conecte este punto con los módulos 1–9.

16. Paso 11 — Evaluación

Crear un conjunto de pruebas.

Mínimo sugerido:

• 3 casos normales.

• 2 ambiguos.

• 2 fuera de alcance.

• 1 error de herramienta.

• 1 error de recuperación.

• 1 caso de seguridad.

SuZuMaCyber Instructor

Nota para el docente: Cada prueba debe tener resultado esperado y resultado obtenido.

Sugerencia didáctica: pedir al estudiante que conecte este punto con los módulos 1–9.

17. Golden Set del proyecto

Crear casos de referencia que permitan comparar versiones.

Ejemplo:

ID | Entrada | Resultado esperado | Resultado obtenido | Estado

El golden set debe permanecer estable durante las mejoras.

SuZuMaCyber Instructor

Nota para el docente: Explica que cambiar el conjunto de pruebas dificulta comparar versiones.

Sugerencia didáctica: pedir al estudiante que conecte este punto con los módulos 1–9.

18. Paso 12 — Observabilidad

Registrar una traza de ejecución:

Entrada

→ Plan

→ Tool call

→ RAG

→ Resultado

→ Validación

→ Salida

Registrar tiempos, estados y errores relevantes.

SuZuMaCyber Instructor

Nota para el docente: El alumno debe poder explicar qué hizo el agente sin adivinarlo.

Sugerencia didáctica: pedir al estudiante que conecte este punto con los módulos 1–9.

19. Paso 13 — Desarrollo por versiones

VERSION 1

Prototipo mínimo.

VERSION 2

Agregar una capacidad.

VERSION 3

Mejorar evaluación y errores.

VERSION FINAL

Seguridad + observabilidad + documentación.

SuZuMaCyber Instructor

Nota para el docente: No intentar construir la versión final en una sola sesión.

Sugerencia didáctica: pedir al estudiante que conecte este punto con los módulos 1–9.

20. Entregable técnico

El proyecto debe incluir:

• código fuente;

• configuración;

• prompts/instrucciones;

• herramientas;

• documentos RAG, si aplica;

• pruebas;

• logs o trazas de ejemplo;

• README;

• instrucciones de ejecución.

SuZuMaCyber Instructor

Nota para el docente: Define una estructura de repositorio clara desde el inicio.

Sugerencia didáctica: pedir al estudiante que conecte este punto con los módulos 1–9.

21. README obligatorio

Debe explicar:

1. Problema.

2. Usuario.

3. Arquitectura.

4. Componentes.

5. Instalación.

6. Configuración.

7. Ejecución.

8. Pruebas.

9. Limitaciones.

10. Mejoras futuras.

SuZuMaCyber Instructor

Nota para el docente: El README debe permitir que otra persona entienda el proyecto sin una explicación oral.

Sugerencia didáctica: pedir al estudiante que conecte este punto con los módulos 1–9.

22. Demostración final

Presentación sugerida:

1. Problema — 1 min.

2. Arquitectura — 2 min.

3. Demostración — 5 min.

4. Evaluación — 2 min.

5. Seguridad y límites — 1 min.

6. Aprendizajes — 1 min.

SuZuMaCyber Instructor

Nota para el docente: La demostración debe mostrar el sistema funcionando, no solamente diapositivas.

Sugerencia didáctica: pedir al estudiante que conecte este punto con los módulos 1–9.

23. Rúbrica de evaluación

Problema y alcance — 15%

Arquitectura — 15%

Implementación — 20%

Integración de IA — 15%

Evaluación y pruebas — 15%

Seguridad y observabilidad — 10%

Documentación y presentación — 10%

SuZuMaCyber Instructor

Nota para el docente: La rúbrica evalúa el proyecto completo; agregar más agentes no otorga puntos por sí mismo.

Sugerencia didáctica: pedir al estudiante que conecte este punto con los módulos 1–9.

24. Checklist de entrega

■ Problema definido

■ Usuario definido

■ Alcance

■ Arquitectura

■ Agente funcional

■ Herramientas/RAG/memoria según necesidad

■ Pruebas

■ Golden set

■ Observabilidad

■ Seguridad

■ README

■ Demo final

SuZuMaCyber Instructor

Nota para el docente: Usa esta lista como revisión previa a la entrega.

Sugerencia didáctica: pedir al estudiante que conecte este punto con los módulos 1–9.

25. Presentación final

SUZU MA ACADEMY

“Construye agentes que no solo respondan: que sepan utilizar información, herramientas y procesos de manera controlada.”

PROYECTO FINAL

SuZuMa Tutor / Agente del estudiante

SuZuMaCyber Instructor

Nota para el docente: Cierra invitando a los alumnos a pensar en el proyecto como su primera pieza de portafolio profesional.

Sugerencia didáctica: pedir al estudiante que conecte este punto con los módulos 1–9.

Anexos

A. Planeación de clase

Módulo / sesión: _____

Fecha: _____

Duración: _____

Objetivo de aprendizaje: _____

Actividad principal: _____

Evidencia esperada: _____

Materiales / recursos: _____

Dificultades observadas: _____

Acuerdos / seguimiento: _____

B. Registro de pruebas del agente

ID	Entrada	Esperado	Obtenido	Estado	Observaciones

C. Checklist de revisión final

- ■ El problema y usuario están definidos.
- ■ El alcance está documentado.
- ■ La arquitectura tiene una justificación.
- ■ Las herramientas tienen contratos y límites.
- ■ El RAG usa fuentes identificables cuando corresponde.
- ■ La memoria/estado tiene una política clara.
- ■ El agente tiene condiciones de finalización.
- ■ Existen pruebas reproducibles.
- ■ Existe un Golden Set.
- ■ Hay trazas o logs suficientes para investigar fallos.
- ■ Los secretos y permisos están protegidos.

■ Existe README y procedimiento de ejecución.

■ La demostración muestra el sistema funcionando.

D. Rúbrica del proyecto integrador

Criterio	Peso	Evidencia
Problema y alcance	15%	Necesidad clara, usuario definido y límites.
Arquitectura	15%	Diseño coherente y justificado.
Implementación	20%	Agente funcional y componentes integrados.
Integración de IA	15%	Uso pertinente de memoria, tools, RAG, planificación o multiagente.
Evaluación y pruebas	15%	Casos reproducibles y resultados.
Seguridad y observabilidad	10%	Controles, trazas, logs y límites.
Documentación y presentación	10%	README, explicación y demostración.

Nota final para el docente

El objetivo del curso no es que el estudiante agregue la mayor cantidad de componentes posibles. El aprendizaje central es saber identificar un problema, diseñar una solución proporcional, integrar capacidades de IA, probarla, observarla, establecer límites y explicar por qué funciona o dónde falla.